AI/ML Assessment Project: Easy Explanation

A simple guide to explain your project in interviews

Overview of the Project

This project is a complete, production-ready AI and Data Engineering system. It is divided into three main problem statements. If anyone asks what you built, you can say: 'I built a 3-part system that includes Machine Learning for predictive maintenance, a high-performance streaming API for log analysis, and an AI Chatbot (RAG) for HR policies.'

Part 1: Machine Learning (Predictive Maintenance)

The Goal: Predict when a machine or sensor is going to fail.

What we did:
- The data was highly imbalanced (only 0.5% of the data were actual failures). This is common in real life.
- We built a robust pipeline using Scikit-Learn that scales the data and handles extreme outliers.
- We trained two models: SMOTE + XGBoost, and a Cost-Sensitive Random Forest.
- We focused on Business Cost, not just accuracy. Since a False Negative (missing a failure) costs $100 and a False Positive (false alarm) costs $1, we mathematically tuned the probability threshold to minimize the financial cost for the company. XGBoost won, saving the business thousands of dollars.

Part 2: Log Analyzer API (Core Python & Memory Safety)

The Goal: Process massive log files (like 50GB) without crashing the server.

What we did:
- The old code read the entire file into memory at once, causing Out-Of-Memory (OOM) crashes.
- We rewrote it using Python Generators (yield) and Context Managers. This means it streams the file line-by-line, keeping the memory footprint at almost zero (O(1) space complexity).

AI/ML Assessment Project: Easy Explanation

A simple guide to explain your project in interviews

- We wrapped this inside a FastAPI backend so users can upload massive files over an API, and the server processes it seamlessly to flag suspicious transactions over $10,000.

Part 3: Advanced AI Systems (HR Policy RAG)

The Goal: Build an AI assistant that answers employee questions based strictly on company HR policies, without hallucinating.

What we did:

- RAG stands for Retrieval-Augmented Generation. Instead of relying on ChatGPT's generic knowledge, we force the AI to read our specific documents.
- We chunked a mock HR policy document and converted the text into embeddings (mathematical vectors) using a local HuggingFace model.
- We stored these vectors in ChromaDB (a vector database).
- When a user asks 'How much maternity leave do I get?', the system searches ChromaDB for the most relevant paragraphs, and sends ONLY those paragraphs to the LLM to generate a factual answer.
- We built a beautiful dark-mode web dashboard to make it easy for non-technical users to interact with both the Log Analyzer and the AI Chatbot.